PES UNIVERSITY, Bangalore

Established under Karnataka Act No. 16 of 2013

Department of Computer Science & Engineering

UE24CS341A : Software Engineering

LAB - 05

Name: Kari Bhavitha	SRN: PES1UG25CS821	Section: F

Static Code Analysis – Issues Found and Fixed

Scores recorded in the original report: Pylint 4.43 → 10.00 | Bandit 5 findings → 0 | Flake8 35 warnings → 0

Issue	Type	Line(s) in original	Description in simple words	How it was fixed
AES in ECB mode with PyCrypto	Security (High)	8, 72–74	ECB can show patterns because the same card data can produce the same encrypted result. PyCrypto is outdated, and the old code could cut off long card numbers.	Changed to AES-GCM with a new random 12-byte nonce each time. Added a function to decrypt the card data and check that it works.
Hard-coded secrets	Security (Low)	14–15	The webhook secret and card key were written directly in the code, so they could be exposed when the code is shared.	Read the secrets from environment variables. For the demo, generate temporary random values if they are not set.
Mako template XSS	Security (Medium)	84–86	The username was added to the HTML without escaping it. Someone could enter HTML that runs unwanted code.	Removed Mako and used html.escape() to safely display the username.
Weak random number generator for coupon codes	Security (Low)	67	The normal random module can make codes that are easier to guess.	Used secrets.choice() to generate less predictable coupon codes.
Using assert for validation	Security (Low)	85	An assert check can be skipped when Python runs in optimized mode, so it should not be used for important validation.	Replaced the assert check with a ValueError.
Unsafe XML parsing	Security	80–82	The old XML parser was not designed to safely handle all untrusted XML input. This can create an XML security risk.	Changed to defusedxml.ElementTree to parse the webhook XML more safely.

Signature comparison not constant-time	Security	78	Using == to compare signatures can potentially reveal information from how long the comparison takes.	Used hmac.compare_digest() to compare the signatures more safely.
Return inside finally block	Bug (Pylint)	58–64	The refund code used finally in a way that could hide errors and leave the refund value unset.	Removed the unnecessary try/finally block and calculated the refund directly.
Refund ignores days used	Logic bug	58–63	The old code returned the full plan price even when the user had already used most of the billing period.	Calculated the refund using the unused days and limited days_used to the allowed range.
Retry loop off by one	Logic bug	52	The loop could try to charge four times even though the limit was three.	Changed the loop condition so it stops at the maximum number of attempts.
Trial period boundary	Logic bug	25	The trial was not marked as over on the 14th day because the comparison used > instead of >=.	Changed the check to use >= TRIAL_DAYS.
Coupon expiry and invalid discount	Logic bug	17–38	The coupon was not accepted on its expiry day. A 150% discount could also create a negative invoice, and using today=None could cause an error.	Allowed coupons through their expiry date, used today's date when no date is given, and rejected discount percentages outside 0–100.
Next billing date	Logic bug	40–41	Adding one to the month can fail in December. Dates like January 31 also need special handling.	Handled the year change and used calendar.monthrange() to keep the day within the new month's length.
Proration uses fixed 30 days	Logic bug	48	Using 30 days for every month can give the wrong upgrade charge. February, for example, has fewer days.	Used the actual number of days in the billing month when calculating the prorated charge.
active_users logic	Logic bug	69–70	The old check treated users with no invoices as active, instead of checking their subscription status.	Counted users as active only when their status is trial or active.
Unused imports and multiple imports on one line	Style	1	Some imports were not needed, and multiple imports on one line made the code less clear.	Removed imports that were not needed and separated imports. The os import remains because it is used for environment variables.
Function names and naming style	Style (Pylint)	Many	Some function names did not follow the usual Python naming style.	Renamed functions to use snake_case.
Missing docstrings	Style (Pylint)	All	The file and functions did not explain clearly what they do.	Added short docstrings to describe the file and functions.

Non-standard Python expressions	Style	21, 29, 70, 86	Some expressions used older or less clear ways of writing Python code.	Used clearer forms such as is None, {}, [], set comprehensions, and normal strings.
Demo runs when the file is imported	Design	107	Importing the file also ran the demo, which is not expected when another program imports it.	Added an if __name__ == '__main__' check so the demo runs only when the file is started directly.
Flake8 formatting warnings	Style	Many	There were blank-line and long-line formatting problems.	Reformatted the code to follow common PEP 8 style rules.